

Lab 06 Web edge and private access

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a private-access diagnosis and traffic-layer diagram. This lab supports LO2. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `PRIVATE_CONFIG`, `DNS_MISMATCH`, `PUBLIC_PATH_REMAINS`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

Firewall and WAF boundaries

Firewall controls network/application egress; WAF inspects HTTP application requests.

Contract: layer, source, destination, host, request_path

Accepted case: WAF blocks a disallowed request at the web edge.

Counterexample: WAF is expected to inspect arbitrary SSH traffic.

Diagnosis: Route non-HTTP controls to the correct network control and preserve WAF request evidence.

Evidence: Show the traffic path and which layer makes each decision.

Private endpoint resolution

Private DNS resolves a service name to the endpoint NIC private address.

Contract: fqdn, resolved_ip, endpoint_ip, public_access

Accepted case: The service resolves to 10.10.3.5 from the training VNet.

Counterexample: It resolves to a public address despite an approved endpoint.

Diagnosis: Check DNS zone linkage, resolver path and public-access configuration.

Evidence: Record DNS output and endpoint approval state; endpoint creation alone is insufficient.

Egress allowlists

The gateway authorises a destination independently of an AI agent's requested action.

Contract: tool, host, port, approved_destination

Accepted case: A report goes to reports.example.test on port 443.

Counterexample: A retrieved document asks for upload to unknown.example.test.

Diagnosis: Deny unapproved destinations and investigate the document as untrusted input.

Evidence: Log destination, policy result and correlation ID without exporting content.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Correct storage-b dns_ip to 10.10.3.6. Its result becomes PRIVATE_CONFIG, but list the real connection and authentication tests still required.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Azure procedure D Inspect web edge and private service access

1. Open the trainer-provisioned Azure Firewall and its Firewall Policy. Record the protected subnet, route path and one approved rule collection. If no firewall is provisioned, review the supplied trainer architecture and record deployment as unverified; do not create a billable appliance without allocation.
2. Open the assigned Web Application Firewall policy. Record its association to the web edge, its mode and one applicable managed/custom rule. Explain that Detection logs and Prevention enforcement are different states.
3. For the allocated storage account, open Networking > Private endpoint connections. Create an endpoint only where the trainer has assigned quota and cost. Select the correct subscription, rg-itsec-<initials>, storage target and required subresource, such as blob.
4. Select the approved VNet/subnet and integrate with the appropriate private DNS zone. Review the target resource and create. Record the endpoint NIC private IP and connection approval state.
5. Open the private DNS zone and its Virtual network links. Confirm the training VNet is linked. From a trainer-approved VM in that VNet, run nslookup <storage-account>.blob.core.windows.net and record the answer chain and final address.
6. Compare the final address with the endpoint NIC address. If it resolves publicly, inspect zone linkage and the resolver path. Do not change public DNS arbitrarily.
7. Review the storage account public network access setting independently. If the exercise requires disabling it, confirm the private path works and get the trainer's go-ahead before changing it.
8. Record one authorised private data operation and a disallowed public-path test where the sandbox supports both. Private DNS alone does not prove authentication or public denial.
9. Save firewall/WAF mode, endpoint approval, DNS and access evidence. Remove only the allocated endpoint and its lab-owned DNS resources after the trainer approves cleanup.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a private-access diagnosis and traffic-layer diagram with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.

- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.